

PEPR Cloud



TARANIS

MODEL, DEPLOY, ORCHESTRATE, AND OPTIMIZE CLOUD APPLICATIONS AND INFRASTRUCTURE

Project and Deliverable Information Sheet

Taranis	Program:	PEPR -- Programmes et équipements prioritaires de recherche
	Project Ref.:	ANR-23-PECL-0008
	Project Title:	Taranis: Model, Deploy, Orchestrate, and Optimize Cloud Applications and Infrastructure
	Keywords:	Infrastructure as Code, DevOps, configuration management, provisioning, orchestration, cluster management
	Project Officer:	Christian Perez
	Project Web Site:	https://pepr-cloud.fr
	Consortium:	Organismes

D2.1 A Survey on Infrastructure as Code	Deliverable ID:	D2.1
	Deliverable Title:	A Survey on Infrastructure as Code
	Deliverable Nature:	Report
	Universal ID:	DOI
	Dissemination Level¹:	PU
	Status:	beta
	Deliverable Leader:	Hélène Coullon
	Contractual Date of Delivery:	2026-03-15
	Actual Date of Delivery:	2025-03-15

Document Control Sheet

Authorship	Written by:	Quentin Guilloteau, Ouadie Khebbab, Eloi Perdereau, Jolan Philippe, Hélène Coullon, Philippe Merle
	Contributors:	
	Reviewed by:	Reviewer
	Approved by:	Christian Perez

Version History

Version	Date	Status	Author	Comments
Alpha	2024-06-12	Draft	Organisme	First Version
Beta	2024-09-25	Draft	Organisme	Core document
V1.0	2025-03-14	Final	Organisme	After Reviews

Copyright notices

© 2024 PEPR Cloud

All trademarks and other rights on third party products mentioned in this document are acknowledged as owned by their respective holders.

¹ The dissemination level are indicated as follows: PU – Public, CO – Confidential, only for members of the consortium (including the Commission Services) CL – Classified, as referred to in Commission Decision 2991/844/EC.

Executive Summary

Infrastructure as Code (IaC) has enriched DevOps practices by enabling the programmatic definition of deployment and management of IT infrastructure. However, the vast spectrum of IaC technologies, from low-level provisioners to high-level cluster managers, has resulted in a fragmented landscape that remains difficult for both practitioners and researchers to navigate. This report provides a systematic classification of the IaC solutions. We propose a survey for classifying IaC technologies based on three distinct categories of classes: (i) Intentions and design goals, characterizing solutions across three primary functional layers: provisioning, configuration management, and cluster management; (ii) Usage and user interfaces, which mostly discusses the language construction and the adopted logic style for the definition of IaC artifacts; (iii) Behavioral and execution models, focusing on the internal mechanism of solutions to process reconfiguration. In this survey, we classified 14 solutions, reflecting the IaC technologies that are currently used by the DevOps community. In contrast to existing taxonomies in the literature, our survey encompasses a broader spectrum of IaC technologies and introduces a more granular classification of their underlying features.

Table of Contents

1. Introduction	4
2. Background	6
2.1. Definitions	6
2.2. History of Infrastructure as Code	7
3. Scope of the survey	8
3.1. Classes	9
3.1.1. Tool intentions and design goals	9
3.1.2. Usage and user interfaces	9
3.1.3. Behavioral and execution models	10
3.2. Considered tools and languages	10
4. Tool intentions and design goals	13
4.1. Tool designed with a primary intent	14
4.1.1. Provisioning	14
4.1.2. Configuration management	14
4.1.3. Cluster management	14
4.2. End-to-end infrastructure management	15
5. Usage and user interfaces	15
6. Behavioral and execution models	18
7. Related work	20
7.1. Studied Infrastructure as Code solutions	21
7.2. Classes for Infrastructure as Code	22
7.3. Quality assurance in Infrastructure as Code	23
8. Conclusion	23
9. Bibliography	23
10. Liste des acronymes, graphiques et tableaux	27
10.1. Liste des acronymes et abréviations	27
10.2. List of Tables	28
10.3. List of Figures	28

1. Introduction

Large distributed software systems are now ubiquitous. A distributed system is ideally constituted of decoupled software entities, often called components. Such a design favors good software engineering properties such as maintainability, flexibility, and separation of concerns. This design also favors the distribution of components across distant machines. In the specific case of utility computing, components are mostly designed as services (or micro-services) that offer interfaces and require external services offered by other components. Examples of widely used APIs (application programming interface) between services are REST (Representational State Transfer²), gRPC³ (remote procedure calls), GraphQL⁴ (note that all of them are based on the HTTP protocol).

²<https://opikanoba.org/tr/fielding/rest/>

³<https://grpc.io/>

⁴<https://graphql.org/>

Once implemented, a distributed software system (i.e., distributed application) is typically deployed on heterogeneous distributed infrastructures. The word infrastructure usually denotes any piece of hardware or software on which systems or applications can be commissioned and executed. Infrastructures can themselves be considered as a set of interacting services offering and using APIs and relying on lower-level pieces of infrastructures. For example, traditional system administration consists in connecting through SSH calls to distant machines and using operating system interfaces to perform actions; provisioning a virtual machine on AWS (Amazon Web Services) or GCP (Google Cloud Platform) consists in REST API calls; creating a pod (container) in Kubernetes consists in CLI (Command Line Interface) calls that themselves perform REST API calls to the *apiserver* component⁵, etc.

Deploying and operating distributed infrastructures through time, as well as distributed applications or software systems running on top of them, is a difficult task. For years, companies worked in silos to address this problem: on one side, application and services developers responsible for coding fast, adding new features, and correcting bugs very often; on the other side, system administrators responsible for the availability, stability, and security of their infrastructures (servers). Due to the incompatibility of their objectives, conflicts arose between these two silos in companies, thereby slowing down the release time. The DevOps concepts (and their role and associated jobs in companies) arose from this problem. Indeed, slow-release processes became naturally incompatible with the Cloud-oriented economic model, where applications are continuously improved and extended while guaranteeing constant availability. The role of a DevOps engineer is to automate a continuous loop between development and operations on infrastructure. In practice, this includes many different aspects. We list some of them amongst others:

1. automatic provisioning of distant infrastructure resources (e.g., servers, virtual machines (VMs), containers, Kubernetes resources),
2. automatic configuration of the provisioned infrastructure resources (e.g., install packages, configure security rules),
3. automatic deployment (i.e., installation or commissioning) of services and applications on the configured resources (e.g., download Docker images, IP and port configuration/integration),
4. automatic monitoring (i.e., observability) of infrastructure resources, services, and applications to automatically detect problems (e.g., performance or security issues, faults),
5. automatic changes (reconfiguration) in infrastructure and services to fix detected issues, this may lead to new provisioning, new configurations, etc., thus creating a loop.

Figure 1 illustrates the workflow of installing and managing a digital infrastructure and applications on top of it. In practice, a DevOps engineer solves each of the above aspects by leveraging and integrating different kinds of DevOps tools. In the above description, one can immediately see the feedback control loop of the MAPE-K model presented by IBM in 2003: (M)onitoring the system, (A)nalyzing the new state to reach, (P)lanning changes to reach this new state, and (E)xecuting this plan.

Amongst the variety of DevOps tools and mechanisms, this document focuses on a specific type, namely Infrastructure as Code, IaC for short. The initial idea behind this term is to consider the management of infrastructures and associated applications as the management of code in usual development teams. To achieve this idea, provisioning, configuring, and operating infrastructure have progressively evolved towards using domain-specific languages to specify infrastructures through code. Hence, infrastructures become easily sharable, testable, versioned, etc.

It appears in practice that IaC languages represent a constellation from which it is difficult to extract a clear taxonomy, frontiers, properties, etc. Giving a clear definition of the word Infrastructure as Code is already difficult. In this document, we propose a detailed taxonomy and analysis of Infrastructure as Code languages to facilitate their comparison and clearly understand their properties.

The rest of this paper is organized as follows. Section 2. gives general definitions about infrastructure as code and presents a running example that will be used throughout the paper. Section 3. presents the scope

⁵<https://kubernetes.io/docs/reference/command-line-tools-reference/kube-apiserver/>

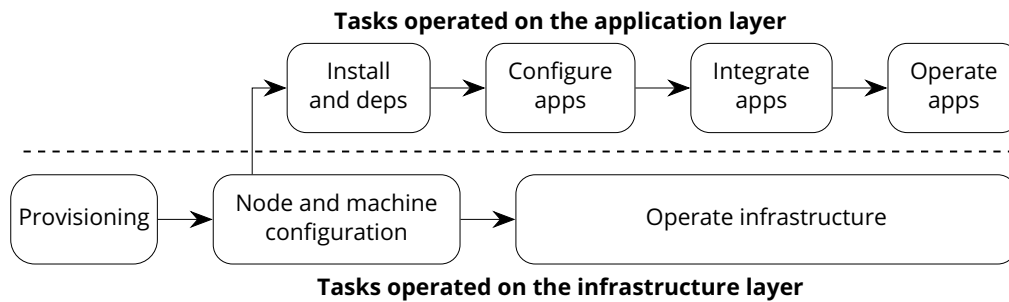


Figure 1: Global overview of tasks operated by infrastructure as code. The tasks operating on the application layer are depicted above the dotted line, while the tasks on the infrastructure layer that host the application are depicted below.

of the survey with an overview of the studied aspects. In Section 4., Section 5., and Section 6., we respectively detail the intention, the usage, and the behavior of Infrastructure as Code solutions. We discuss the position of our survey in Section 7. before giving concluding remarks in Section 8..

2. Background

2.1. Definitions

Many existing definitions related to infrastructure management exist across the literature. To ensure a consistent understanding and avoid confusion for the reader throughout the rest of the document, a set of definitions is provided below.

Definition 1 (Infrastructure). An infrastructure is the set of underlying hardware, software, equipment, etc., that a computer system needs to operate. It can be interpreted at different levels. For instance, machines are physical pieces of infrastructure required to run operating systems; Infrastructure as a Service is the infrastructure required to run Software as a Service, etc. From [1], infrastructure refers to the software, platform, or hardware that delivers or deploys applications to production. Note that, in this paper, we consider all types of infrastructure that offer software interfaces to be controlled. This includes operating systems, containers, virtual machines, and distant virtual infrastructure resources (networks, VMs, etc.), for instance. An infrastructure piece is called a *resource* or a *component*.

Definition 2 (Resource). A resource is an abstract term that represents any piece of software deployed on top of an infrastructure. As an infrastructure is a multi-level term, a resource can take various forms, e.g., operating systems, virtual networks, micro services in a container.

Definition 3 (Cloud computing). The Cloud paradigm is a key driver behind the development of Infrastructure as Code. It promotes that infrastructure (at different levels) should be offered as utility services: easily accessible (i.e., through the internet for the Cloud), with quality guarantees (e.g., availability), and associated with a pay-as-you-go economic model. Amongst others, with increasing levels, the main services of Cloud computing are categorized as Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS). The Cloud providers are the actual maintainers of the utility computing infrastructure. In practice, Cloud providers manage very large data centers shared amongst their users. The fundamental mechanism at the root of Cloud computing is the virtualization of computing, network, and storage resources.

Definition 4 (DevOps). DevOps is a set of practices and methodological principles that aim to improve collaboration and integration between development (Dev) and operations (Ops) teams. It promotes a unified approach to the software delivery lifecycle, emphasizing automation, monitoring, and continuous feedback. DevOps engineers, that we could also call Infrastructure as Code *users*, are responsible for implementing and maintaining Continuous Integration (CI) and Continuous Deployment (CD) pipelines, ensuring that code changes are reliably built, tested, and delivered to production environments, including the required infrastructure.

Definition 5 (Configuration). In the DevOps and infrastructure management community, a *configuration* is broad and represents how various infrastructure and application resources are set up. It can include a set of system configurations (through files or commands) of virtual or physical infrastructure resources, a set of infrastructure or application resources to install and commission, a set of key/value pairs where values are chosen among offered options, and some dependencies between infrastructure resources, either hidden within key-value pairs or explicitly specified through the IaC language.

Definition 6 (State). A *state* of an infrastructure resource consists of its specific configuration along with runtime information that reflects dynamic aspects of the resource (e.g., living runtime data, active processes, resource usage, or other observable metrics). The state of an infrastructure is the union of the states of each of its constituent resources.

Definition 7 (Reconfiguration). According to the configuration definition, a reconfiguration consists of changing the configuration of running and under usage various resources. It allows infrastructures to adapt in response to requirements, environmental conditions, or objectives. This term is not used a lot in the IaC community, where *maintenance*, *change*, or *deployment* (not only initial) is preferred. In the academic literature, this term represents a field in Component-Based Software Engineering (CBSE) that mostly focuses on changing the component instances involved in a distributed software system, as well as their topology, while not interrupting its services. A *reconfiguration* also refers to a transient state between the current state and the target state.

Definition 8 (Deployment). A *deployment* is a particular reconfiguration focused on transitioning from an inactive state to an operational state. It encompasses resource allocation, initialization, and activation of the necessary runtime parts of the system to meet a defined configuration, thus making a system or application ready for use. It is also important to note that in some cases, deployment is considered to be a given reconfiguration, so that a system undergoes a succession of deployments over time.

Definition 9 (Provisioning). The *provisioning* defines the process of allocating resources that a user (i.e., DevOps) requires through API calls. Provisioning approaches were originally proposed by Cloud providers (e.g., virtual machines, virtual network) to enable automation of distant resource management.

Definition 10 (Configuration management). The *configuration management* refers to the precise specification, enforcement, and maintenance of the desired configuration of system resources within an infrastructure. Configuration management is used to configure already provisioned infrastructure resources.

Definition 11 (Orchestration). The *orchestration* process automates the lifecycle coordination and management of interacting resources within an infrastructure. An orchestrator typically constantly (or frequently) checks the state of infrastructure resources to reconcile their current state with the desired state. This enables advanced controlling techniques such as automatic scaling, self-healing, etc. In practice, tools that are usually called orchestrators consist of small operating systems that manage an on-premise underlying infrastructure (i.e., servers) and deploy higher-level resources on top of it. For this reason, and as done in the literature, we prefer the term *cluster managers* [2, 3].

Definition 12 (Infrastructure as Code). Infrastructure as Code (IaC) is a paradigm for orchestrating the *deployment lifecycle* of software systems through code-based abstractions [4, 5]. It treats infrastructure and deployment processes as software artifacts. These IaC artifacts enable infrastructure tasks to be versioned, reviewed, tested, and integrated within software delivery pipelines (i.e., CI/CD) [6, 7], bridging the gap between application development and operations (see Section 2.2.). IaC covers the deployment lifecycle, automating three key capabilities: *provisioning*, *configuration management*, and *cluster management* [4, 8].

2.2. History of Infrastructure as Code

The evolution of Infrastructure as Code parallels the increasing complexity of distributed systems management. Figure 2 proposes a timeline of Infrastructure as Code. In the early 1990s, maintaining consistency in systems became harder due to the rise of heterogeneous environments (e.g., different operating systems) [9]. This challenge necessitated automated configuration management, addressed by the release of CFEngine in 1993 [10]. A significant paradigm shift occurred in 2006 with the launch of Amazon Elastic Compute Cloud

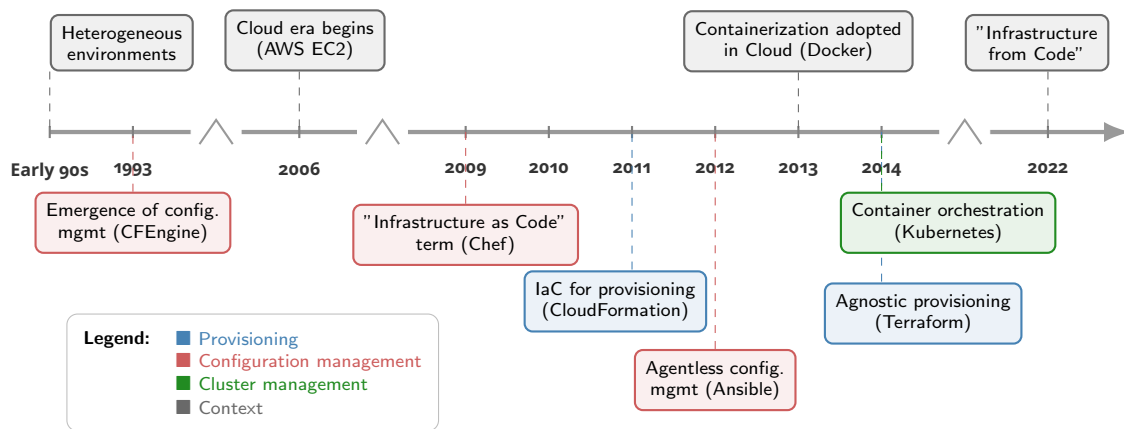


Figure 2: Timeline of Infrastructure as Code capabilities appearance

(or simply AWS EC2)⁶, which introduced programmable infrastructure and marked the beginning of the Cloud era⁷. Consequently, the term "Infrastructure as Code" was popularized in 2009 with the release of Chef [11], while subsequent tools for configuration management like Ansible evolved the paradigm in 2012 by introducing agentless architectures [12]. The focus expanded from configuring servers to provisioning them, leading to the development of vendor-specific tools such as AWS CloudFormation in 2011 [13]. In 2014, new provisioning cloud-agnostic solutions were proposed (e.g., Terraform [14]). Finally, the widespread adoption of containerization following the release of Docker in 2013 [15] drove the need for advanced container orchestration. This demand was addressed in 2014 with Kubernetes [16]. Nowadays, research on Infrastructure as Code remains an active field. The current challenges reside in writing correct and optimal Infrastructure as Code artifacts. The *Infrastructure from Code* (IfC) [17] paradigm was introduced around 2022, and design tools that automatically infer the necessary infrastructure directly from the application's source code and its logical requirements.

3. Scope of the survey

This survey aims to categorize IaC solutions by their features. Certain aspects of IaC practices and specific tool categories are intentionally excluded from this survey. We do not cover auxiliary support for IaC, such as third-party linters for code smells, Language Server Protocols (LSP) for development, or network configuration (e.g., Netplan⁸).

Cloud-based hosting solutions (e.g., HCP [18]) and package management systems (e.g., opam, pip) used for post-deployment library installation are beyond our scope. We exclude Infrastructure from Code (IfC) [17] approaches where resources are defined within application logic but not explicitly managed through a dedicated deployment workflow. To ensure relevance to modern DevOps practices, we exclude deprecated or unmaintained solutions. This includes tools officially deprecated by vendors in favor of industry standards (e.g., Google Infrastructure Manager [19]) and legacy projects that have seen no active maintenance for over a decade (e.g., SmartFrog [20]). We exclude tools primarily focused on the theoretical design of IaC scripts or those originating from purely academic research (e.g., ConfSolve [21], BtrPlace [22], EnosLib [23], or Concerto [24]). Niche solutions with limited industry adoption (e.g., (R)?ex [25]) are also excluded. Finally, we exclude tools that locally configure a given machine, such as package managers (e.g., Nix) and container managers (e.g., Docker Compose), and instead focus on the management of distant infrastructure resources.

⁶<https://aws.amazon.com/ec2>

⁷<https://greylock.com/greymatter/jerry-chen-the-evolution-of-cloud/>

⁸<https://doc.ubuntu-fr.org/netplan>

3.1. Classes

We classify the surveyed IaC technologies by decomposing their characteristics into three distinct perspectives: the *why*, the *how*, and the *what*. First, Section 4., entitled “tool intentions and design goals”, addresses the *why*, categorizing tools based on their intended scope, and the specific infrastructure layers they target. Second, Section 5., entitled “usage and user interface”, addresses the *how*, focusing on the human-centric aspects of language design, syntax, and user interaction with the system. Finally, Section 6., entitled “behavioral and execution models”, addresses the *what*, describing the internal mechanisms regarding execution flow and state management. In the following, we detail each classification criterion associated with the above decomposition. Furthermore, an overview of all studied classes is given as a feature diagram in Figure 3.

3.1.1. Tool intentions and design goals

In this first group of classes, we aim to present features to understand the IaC tool’s intentions. We identified three classes described below.

Class 1 (Capability). We organize the spectrum of Infrastructure as Code into three capabilities that evolved in response to changing operational needs and resource access mechanisms (see Figure 2):

- *Provisioning* defines the process of allocating resources that an application requires through API calls. Provisioning approaches were originally proposed by Cloud providers (e.g., virtual machines, virtual network) to enable automation of distant resource management. Tools were specific to each Cloud provider. Then, the provisioning concept has been extended to any Infrastructure API calls (i.e., Cloud providers, containers, and cluster managers).
- *Configuration management* refers to the precise specification, enforcement, and maintenance of the desired configuration of system resources within an infrastructure. Configuration management is used to configure already provisioned infrastructure resources on which the user has the right to modify the configuration. Configuration management is initially close to scripting solutions where system-level commands are executed on infrastructure resources.
- *Cluster management* automates the lifecycle management of computing elements, from infrastructure resources to application-level components, by ensuring continuous reconciliation between the desired and current states. Cluster managers apply changes using autonomous loops: monitoring detects drifts or events, and control logic triggers appropriate reconfiguration actions across the infrastructure or application stack.

Class 2 (Adaptation). Unlike static provisioning, cluster management proposes a dynamic evolution of the infrastructure. This capability differs across IaC solutions. *Self-healing* addresses the automatic recovery from service failures. *Autoscaling* manages the so-called horizontal (number of resources) or vertical (size of resources) scaling automatically. Horizontal scaling adjusts the number of active units (e.g., virtual machines or containers) and their individual power (e.g., CPU, RAM) based on real-time demand. *Scheduling* is the automatic planning and placement of components across the available infrastructure. It involves a decision-making process where constraints (e.g., hardware requirements) are used to determine reconfiguration actions.

Class 3 (Target). *Dedicated* (or provider-specific) technologies are solutions developed for a single cloud provider to manage resources exclusively within its own ecosystem. These tools are often deeply integrated with the provider’s backend, allowing for immediate support of new features and internal state management, but they are limited to a single target environment. Conversely, *agnostic* (or cross-provider) technologies are designed to interface with multiple, heterogeneous providers through a unified workflow.

3.1.2. Usage and user interfaces

In this second group of classes, we aim to present features to describe how IaC tools are used in practice and what the user interfaces are. We identified three classes described below.

Class 4 (Construction). A *Domain-Specific Language (DSL)* is a computer language with a limited scope, designed to solve a particular problem within a specific application domain (e.g., Bicep for provisioning). DSLs are contrasted with *General-Purpose Languages (GPLs)*, which are designed to solve various problems across multiple domains (e.g., Python, Java). A DSL is usually easier for users (not taking the learning curve of a new language into account) because more adapted to the targeted purpose, but less flexible and powerful than a GPL.

Class 5 (Logic style). From a user perspective, an *imperative* (or procedural) solution requires code that describes infrastructure management through explicit order of operations to be executed. In contrast, a *declarative* IaC solution asks the user to specify the desired state of the infrastructure, not how to get to this desired state. The associated actions are synthesized (or derived) from the specification, which also offers additional safety guarantees in complex cases and if the tool is well specified (ideally formally) and coded.

3.1.3. Behavioral and execution models

Finally, in this third group of classes, we aim to classify the ways IaC tools behave when executing code, e.g., their semantics and architectural choices. We identified five classes described below.

Class 6 (Client Agent). The client agent class defines whether the solution requires an agent (i.e., *agent-based*) or not (i.e., *agentless*). It specifies if a specific *client agent* must be installed and running on the reconfigured nodes (the target infrastructure) to execute the management tasks.

Class 7 (State management). A *stateful* IaC solution maintains a persistent view of the infrastructure state. This stored state can be updated automatically through continuous monitoring, ensuring real-time consistency, or in a discrete manner through explicit calls. In contrast, a *stateless* system does not keep track of the infrastructure state.

Class 8 (Communication). In a *push-based* strategy, a central controller initiates the connection to the managed nodes to send instructions. Conversely, a *pull-based* strategy relies on agents residing on the managed nodes that reach out to a central master to request their intended configuration.

Class 9 (Reconciliation). The reconciliation process defines the mechanism and the frequency at which the tool synchronizes the *current state* of the infrastructure with the *desired state*. This can be performed:

- *manual*: The user must explicitly trigger the reconciliation (e.g., via a CLI command).
- *event*: The reconciliation is triggered by a monitored threshold or a specific system event (e.g., CPU usage).
- *periodic*: The tool automatically checks and enforces the state at regular time intervals.
- *programmable*: The user can define custom logic to determine when reconciliation should occur.

3.2. Considered tools and languages

The studied tools, in alphabetical order, are the following.

- **Ansible** [12], developed by Michael DeHaan and first released in 2012, is an open-source automation tool licensed under GNU GPLv3. It is one of the most widely used infrastructure as code tools, now sponsored by Red Hat (which acquired Ansible in 2015) and driven by a large open-source community (over 5,000 contributors on GitHub). The Ansible Python package is downloaded over 5 to 8 million times per month. The project sees active development ($\approx$ 50k commits/year).
- **CloudFormation** [13], launched by Amazon Web Services (AWS) in 2011, is a proprietary infrastructure as code service allowing users to provision AWS resources. It is fully maintained by AWS as part of its private cloud platform. CloudFormation is widely used by AWS customers for automating infrastructure deployments.



Figure 3: Diagram of studied features in IaC technologies

- **Azure Resource Manager** [26] (ARM), introduced by Microsoft in 2014, is a proprietary Azure's native deployment service. ARM is maintained by Microsoft and tightly integrated into the Azure ecosystem (e.g., Azure Portal). The solution is used by Azure customers as the default infrastructure as code mechanism for Azure. While ARM is not a community-driven project, Azure QuickStart Templates repository has over 1,300 contributors submitting example ARM templates. In 2020, Microsoft released Bicep, a new DSL dedicated to ARM for easing ARM usage and growing its user community.
- **CFEngine** [10], created in 1993 by Mark Burgess, is a lightweight configuration management tool (one of the earliest in this domain). It is open-source (GPLv3 for the community edition) with a commercial enterprise version offering enhanced features. CFEngine is currently maintained by Northern.tech AS (the company formerly known as CFEngine AS). Although its popularity peaked in the early 2000s, it is still used in some organizations, valued for its low resource footprint and policy-based automation. The community is smaller today compared to newer tools, but it remains active.
- **Chef** [11], developed by Opscode (later Chef Software, Inc.), and released in 2009, is a Ruby-based configuration management tool licensed under Apache 2.0. Chef was the first to introduce the concept of infrastructure as code via cookbooks and recipes. It is now maintained by Progress Software (which acquired Chef in 2020) with open-source Chef and a premium enterprise dashboard: Chef Automate. Many large enterprises (e.g., Meta, General Motors) used Chef extensively in the 2010s, but its usage has declined relative to more recent and simpler tools (e.g., Ansible).
- **Heat** [27], part of the OpenStack project since 2012, is an orchestration engine that allows users to define and automate the deployment of cloud applications and infrastructure using HOT (Heat Orchestration Template). Heat is open-source (Apache 2.0) and is maintained by the OpenStack community (now under the OpenInfra Foundation).
- **Juju** [28], developed by Canonical and first released in 2011, is a model-driven application and service orchestration tool. Juju is open-source (Apache 2.0) and primarily maintained by Canonical, with an open community of contributors. The Juju project has around 2.5k stars on GitHub.
- **Kubernetes** [16], originally designed by Google and open-sourced in 2014 (now under CNCF governance, Apache 2.0 license), is the standard for container orchestration. It has one of the largest open-source communities in the world. In 2024, Kubernetes has over 74,000 contributors and more than 314,000 commits, and the project has around 100k stars on GitHub. Kubernetes is maintained by the CNCF Kubernetes SIGs and Steering Committee, with contributions from thousands of engineers across many companies (e.g., Intel, Google).
- **Nomad** [29], developed by HashiCorp and released in 2015 under MPL 2.0 (now transitioned to HashiCorp's BSL in 2023), is a workload scheduler and cluster orchestrator that can deploy containers as well as non-containerized applications. Nomad is open-source (with a source-available license since 2023) and maintained primarily by HashiCorp, with community contributions (about 1,500 contributors and 15k stars on GitHub).
- **Pulumi** [30], introduced in 2018 by Pulumi Corporation (Apache 2.0 license), is an infrastructure as code framework that enables infrastructure definition using general-purpose programming languages (e.g., Python, TypeScript). Pulumi is maintained by Pulumi Corp with a rapidly growing open-source community. In 2024, the Pulumi GitHub project has more than 20k stars. The project counts thousands of contributors and more than 100 million downloads by more than 170,000 developers worldwide.
- **Puppet** [31], created by Luke Kanies in 2005 and inspired by ISConf3 [32], is a declarative configuration management tool that was released under the GPL and later Apache 2.0 (since v2.7). Puppet is maintained by Puppet, Inc. (now a business unit of Perforce since its acquisition in 2022) and has both an open-source version and a commercial Puppet Enterprise edition. Puppet's usage held an estimated 14-15% share of the configuration management market.

- **SaltStack** [33], or simply Salt, introduced in 2011 by Thomas S. Hatch under the Apache 2.0 license, is an open-source configuration management and remote execution platform known for its event-driven automation model. The SaltStack company was acquired by VMware in 2020, and subsequently, VMware's assets (including Salt) fell under Broadcom in 2023. Salt has a large community (nearly 5,000 total contributors and around 15k stars on GitHub).
- **Terraform** [14], developed by HashiCorp and first released in 2014, is the standard declarative IaC tool that allows provisioning of infrastructure across numerous cloud and on-prem providers via a unified HCL language. Originally open-sourced under MPL 2.0, HashiCorp transitioned Terraform to its BSL license in 2023, which led to the creation of a community fork (OpenTofu [34]). The Terraform GitHub repository has thousands of contributors and tens of thousands of stars, and the broader HashiCorp Terraform ecosystem (providers, modules) involves thousands more.
- **TOSCA** [35] (Topology and Orchestration Specification for Cloud Applications) is an OASIS-open standard, first released in 2013, that defines a portable description for the topology of cloud applications and their lifecycle management processes. Unlike the other solutions, TOSCA is a specification rather than an implementation – it is maintained by the OASIS TOSCA Technical Committee, which includes contributors from major companies (e.g., IBM, Cisco, Huawei). TOSCA is available as a series of published standards (recently updated to TOSCA 2.0 in 2025).

In the following three sections, these tools will be compared according to the presented classes.

4. Tool intentions and design goals

This section categorizes infrastructure as code solutions based on their *primary intent*, although many modern tools have overlapping capabilities. We distinguish three main categories: *provisioning*, which manages the lifecycle of physical or virtual resources; *configuration management*, which handles the internal state and software stack of those resources; and *cluster management*, which coordinates service relationships and container deployment. While a tool may belong primarily to one category, it may provide *additional capabilities* from others to support a broader workflow. Table 1 summarizes the intentions of infrastructure as code solutions, including their capacity and their target. We comment in details this table in the following.

Technology	Provisioning	Conf. Mgmt	Cluster. Mgmt	Adaptation	Target
Azure Resource Manager	✓✓	✓	✓	AS, Sch	Dedicated (Azure)
CloudFormation	✓✓	✓	✓	AS, Sch	Dedicated (AWS)
Heat	✓✓	✓	✓	AS, Sch	Dedicated (OpenStack)
Pulumi	✓✓	✓	✗	✗	Agnostic
Terraform	✓✓	✓	✗	✗	Agnostic
Ansible	✓	✓✓	✗	✗	Agnostic
CFEngine	✓	✓✓	✗	✗	Agnostic
Chef	✓	✓✓	✗	✗	Agnostic
Puppet	✓	✓✓	✗	✗	Agnostic
SaltStack	✓	✓✓	✗	✗	Agnostic
Kubernetes	✓✓	✗	✓✓	AS, Sch, P	Agnostic
Nomad	✓✓	✗	✓✓	AS, Sch	Agnostic
Juju	✓✓	✓✓	✓✓	AS, Sch	Agnostic
TOSCA	✓✓	✓✓	✓✓	P	Agnostic

Table 1: Capabilities of infrastructure as code tools: provisioning, configuration management, and cluster management (AS = Autoscaling, P = Programmable, Sch = Scheduling). Symbols indicate whether a capability was originally intended (✓✓), later added (✓), or unsupported (✗).

4.1. Tool designed with a primary intent

4.1.1. Provisioning

Provisioning resources on dedicated infrastructure. Cloud providers often offer native IaC solutions tailored specifically to their own ecosystems. Azure Resource Manager (ARM) is dedicated to provisioning infrastructure within Microsoft Azure, instantiating resources such as `Microsoft.Network` or `Microsoft.Compute` objects. Similarly, AWS CloudFormation (CFN) targets the Amazon Web Services (AWS) ecosystem to provision components like `AWS::EC2` resources. While the primary intent of both tools is the provisioning of resources, they also orchestrate their deployment workflow, managing dependencies to ensure the correct order of operations (i.e., scheduling). Besides, users can define autoscaling policies that automatically adjust compute capacity (e.g., Virtual Machine Scale Sets in Azure and Auto Scaling Groups in AWS) in response to real-time workload demands. Finally, they also support basic configuration tasks by enabling the execution of boot-time scripts for software installation and service setup on compute instances during deployment.

Provisioning resources through OpenStack services with Heat. Heat is an orchestration engine for the OpenStack ecosystem, designed to provision infrastructure by interacting with underlying services (rather than directly manipulating the infrastructure). It manages calls to service APIs (e.g., Nova for compute, Neutron for networking) to instantiate resources. Heat focuses on application orchestration: it allows users to model multi-tier applications as a single manageable unit known as a “stack”. Heat manages the lifecycle of this stack, resolving dependencies between services to ensure the correct deployment order (i.e., scheduling). It also supports dynamic scalability via `AutoScalingGroups` based on monitored metrics (e.g., CPU usage) to automatically scale the number of provisioned resources.

Cross-provider provisioning with Terraform and Pulumi. Terraform and Pulumi are provider-agnostic IaC solutions designed to manage infrastructure across diverse ecosystems (e.g., Amazon Web Services, Microsoft Azure, and Google Cloud Platform). Their primary intent is to provision resources through extensible sets of *providers*, which act as interfaces to specific target APIs. This allows users to define and instantiate typed resources specific to each cloud (e.g., `aws_vpc` or `azurerm_network_interface`) within a unified workflow. Although primarily designed for provisioning, Terraform and Pulumi can also invoke setup actions via remote command execution to perform lightweight configuration management.

4.1.2. Configuration management

Ansible, CFEngine, Chef, Puppet, and SaltStack are provider-agnostic solutions designed for the configuration management of heterogeneous systems. Unlike the previously discussed tools, which focus on creating infrastructure, the primary intent of these solutions is to automate the internal setup and maintenance of compute resources—ranging from physical servers to virtual machines and cloud instances. They function by enforcing a desired state for the operating system and applications, managing entities such as packages, configuration files, users, and services. Although their focus is configuration, these tools have additional capabilities for infrastructure provisioning. Chef and Puppet use specific providers to interface with cloud APIs (e.g., AWS, OpenStack) for creating resources. As an alternative solution for provisioning, Terraform can be integrated with Chef and Puppet using *Chef provisioner*⁹ and *Puppet provisioner*¹⁰, which install and configure the respective clients on newly provisioned virtual machines. Similarly, Ansible and SaltStack include extensive module libraries that allow users to provision cloud infrastructure directly.

4.1.3. Cluster management

Docker Swarm, Kubernetes, and Nomad are considered orchestrators (as Heat, for instance) as they intend to continuously reconcile a declarative desired state of resources with the actual real state on machines. In the

⁹<https://www.hashicorp.com/en/partners/tech/chef>

¹⁰<https://www.hashicorp.com/en/partners/tech/puppet>

literature, these tools are also referred to as cluster management tools, which we think is a good description of their features. Indeed, they intend to place and maintain through time resources of various types on a cluster of machines (physical or virtual). Docker Swarm is fully dedicated to the placement and management of a Dockerized software stack and is not detailed in this survey, as no longer maintained. Kubernetes is a bit less specific to Docker (e.g., it can also use other container technologies) but is still specific (in its default form) to containerized software stacks. Finally, Nomad is more open and provides built-in resources to orchestrate virtual machines or binaries in machines, for instance. However, Kubernetes has the particularity of being programmable and extensible. New types of resources and associated controllers can be added by the mean of “custom resources” and “custom controllers” (or operators). Hence, if built-in resources of Kubernetes are dedicated to containerized environments, custom resources can be added, for instance, to deploy virtual machines orchestrated by Kubernetes. This is, for example, the purpose of Kubevirt. Both Nomad and Kubernetes are equipped with features to dynamically and automatically scale the number of primitive resources (e.g., pods, containers) according to rules, for instance, related to CPU usage. As cluster management tools are responsible for placing resources on machines, they also offer dynamic scheduling and placement policies. In addition of cluster management features, Nomad and Kubernetes offer a language to provision declaratively various types of resources (e.g., pods, autoscalers, virtual machines, jobs). Hence, they can also be considered as provisioning tools, not for Cloud resources, but for dedicated (and custom in the case of Kubernetes) resources.

4.2. End-to-end infrastructure management

TOSCA and Juju enable an end-to-end management of infrastructure, that is, manage both the application management and the underlying infrastructure. TOSCA provides a standard model for the complete structure of a system, defining both the infrastructure elements and the software components alongside their dependencies. Its primary purpose is to offer a portable definition of the system’s lifecycle and operational workflows, which are then executed by compatible third-party orchestrators [36], including *Cloudify*, *OpenTOSCA*, *Unfurl*, *xOpera*, and *Yorc*. In contrast, Juju is an active orchestration engine designed for model-driven operations. Its primary capacity is to manage applications and their integration rather than just the underlying resources. Juju automatically provisions the necessary compute infrastructure (on any major cloud or bare metal) required to host the defined services, and subsequently manages their configuration, scaling, and interconnection.

5. Usage and user interfaces

End-users use the primary interface of the tool to define and manage their infrastructure through scripts or configuration files. These users typically operate at a high level of abstraction. Table 2 summarizes language aspects, such as the specific language used and its logic style. All the studied IaC technologies offer opportunities for extension, allowing developers to write their custom logic (e.g., providers, modules), but this aspect is not discussed in the current survey.

Technology	Language construction		Logic style
Azure Resource Manager	DSL	(Bicep)	Declarative
CloudFormation	DSL	(YAML)	Declarative
Heat	DSL	(HOT)	Declarative
Pulumi	GPL	(Python, JS/TS, Go, C#)	Declarative
Terraform	DSL	(HCL)	Declarative
Ansible	DSL	(YAML + Jinja + ini)	Hybrid
CFEngine	DSL	(CFEngine policy lang.)	Declarative
Chef	GPL	(Ruby)	Imperative
Puppet	DSL	(Puppet lang.)	Declarative
SaltStack	DSL	(YAML + Jinja)	Declarative
Kubernetes	DSL	(YAML)	Declarative
Nomad	DSL	(HCL)	Declarative
Juju	DSL + GPL	(YAML + Bash, Python, Ruby)	Hybrid
TOSCA	DSL	(YAML)	Declarative

Table 2: Characteristics of technologies from an end-user perspective.

Declarative provision of resources All solutions primarily designed for provisioning are declarative, as users define the desired state rather than the execution steps. Tools like Terraform, Azure Resource Manager, and Heat use, respectively, HCL (Hashicorp Configuration Language), Bicep, and HOT (Heat Orchestration Template) as DSLs to provide syntax for declaring resource needs and some conditional logic. For instance, Bicep proposes a `customData` field to inject initialization scripts at startup.

While CloudFormation and Heat use YAML schemas (preferring YAML over JSON for readability), Pulumi adopts GPLs (e.g., Python, JavaScript, TypeScript, Go, C#) to offer developers maximum expressivity. Regardless of the syntax, the underlying engine constructs a dependency graph from the manifest to orchestrate API calls in the correct order.

Despite the declarative foundation, end-users can introduce a slight imperative aspect to the management of mutability and updates. By default, provisioners attempt in-place updates; however, users can force immutable behavior in which resources are destroyed and recreated. The mechanism for this varies across the ecosystem: Terraform uses the `replace_triggered_by` lifecycle meta-argument or CLI flags; Pulumi offers `deleteBeforeReplace` or `replaceOnChanges`; and CloudFormation relies on the specific `Replacement` status defined in the resources specifications. Similarly, Heat allows marking fields as non-updatable, whereas TOSCA achieves this by altering the transition logic to skip the modify state.

Declaring policies for configuration management CFEngine, Puppet, and SaltStack all follow a declarative approach, focusing on specifying the final state of the infrastructure.

In CFEngine, end-users specify a desired configuration of system entities (e.g., services, files, users) with a specialized DSL within policy files (with an `.cf` extension). Users write “promises” that act as constraints to be enforced by the engine. Similarly, users of Puppet and SaltStack describe their desired configurations using their respective DSLs within Puppet manifest files (with a `.pp` extension) or Salt State files (with an `.sls` extension). In Puppet, users declare resources, such as files, packages, services, users, and cron jobs (i.e., scheduled tasks), as persistent promises to be maintained by the agent. In SaltStack, the approach is similar; users define states that ensure specific configurations are present, often leveraging Grains (local system inventory) and Pillars (global variables) to customize the deployment.

CFEngine’s DSL supports a core set of primitive types, including strings, integers, real numbers, booleans, and lists. In Puppet, expressions are defined with a rich, and strictly typed, system that supports integers, strings, callables, patterns, and custom data types. SaltStack relies on the Jinja2 templating engine, which allows for powerful dynamic logic and variable interpolation within its YAML-based state files.

By default, these configurations follow a top-to-bottom resolving order, but users can establish complex dependencies between resources to control execution flow. This is achieved either explicitly through specific

options (e.g., `require` keyword available in both Puppet and SaltStack) or implicitly by writing expressions that refer to other resource values.

Procedure of configuration management operations Ansible and Chef follow an imperative execution style, focusing on the sequence of operations required to reach a desired state.

Ansible runs *playbooks* written in YAML. A playbook executes a list of *plays* on a set of distant hosts. Finally, Ansible managed nodes are specified in an inventory file (typically with an `.ini` or `.yaml` extension) that maps host addresses to groups. A play typically executes a list of tasks, or a list of reusable sets of tasks known as *roles*. Within a task, parametrizable *modules* are invoked. This results in a sequential imperative list of tasks to execute which is specified by the user. However, individual modules are mostly declarative in their usage, which makes us qualify Ansible as hybrid. Chef core logic relies on Ruby. Chef *recipes*, grouped into *cookbooks*, use standard Ruby code to define an execution path, often employing `execute` commands to run shell instructions. Chef calls pre-defined *resources*, designed to operate a particular job. Ansible and Chef have a very similar way to define tasks to operate: an Ansible playbook is equivalent to a Chef cookbook, while Ansible plays correspond to Chef recipes, and Ansible modules align with Chef resources.

Ansible's high-level execution flow inherently makes it procedural, while Ruby, as an adopted language, makes Chef fundamentally imperative. While Chef uses pure Ruby for coding, Ansible expressions and variables leverage Jinja2.

About cluster management usage Both Kubernetes and Nomad expose a declarative programming style by means of YAML manifests and HCL (DSL from Hashicorp) files. These files specify the required resources and their configuration. In Kubernetes, resources can be piled, meaning that some resources are responsible for the management of lower-level resources. For instance, the primitive *pod* resource of Kubernetes (i.e., a set of containers to deploy on the same node) is unlikely to be directly required in a manifest but created and handled through *replicaset* or *deployment* resources. Unlike Nomad, Kubernetes is extensible. Indeed, custom resources and their associated custom controllers (or operators) can be added to Kubernetes. A resource definition is specified in YAML. For instance, Kubebuilder¹¹ is a tool to facilitate custom resources and operators implementation in Go.

Modeling end-to-end orchestration With Juju and TOSCA, users provide a high-level modeling and lifecycle orchestration. Juju uses packages, named *charms*, containing the operations code and metadata to model applications and their relations. End-users describe these deployments using YAML-based metadata files. Besides, they can write additional scripts with diverse GPL (e.g., Bash, Python, Ruby) to define notification controllers (named *hooks*) and reactive *actions* to provide event-based orchestration behavior (see 6.). Similarly, TOSCA follows a "blueprint" approach (i.e., agnostic relationship-centric specification), and provides a standardized YAML-based DSL for describing the topology of cloud applications as a graph of nodes and relationships. The language allows for the definition of *node types*, *capabilities*, and *requirements*, focusing on the inter-connectivity of services. Contrary to any other infrastructure as code solution, TOSCA only provides a standard modeling grammar, and its usage really depends on the specific orchestrator implementation (e.g., *Cloudify*, *OpenTOSCA*).

¹¹<https://book.kubebuilder.io/>

6. Behavioral and execution models

Technology	Agent	State	Communication	Reconciliation
Azure Resource Manager	✗	✓	Push-based	Manual; event
CloudFormation	✗	✓	Push-based	Manual; event
Heat	✗	✓	Push-based	Manual; event
Pulumi	✗	✓	Push-based	Manual
Terraform	✗	✓	Push-based	Manual
Ansible	✗	✗	Push-based	Manual
CFEngine	✓	✓	Pull-based	Periodic
Chef	✓	✓	Pull-based	Manual; periodic
Puppet	✓	✓	Pull-based	Periodic
SaltStack	✓	✓	Hybrid	Manual; periodic; event
Kubernetes	✓	✓	Pull-based	Event
Nomad	✓	✓	Pull-based	Event
Juju	✓	✓	Push-based	Manual; periodic; event
TOSCA	✓	✓	Depends	Depends

Table 3: Execution models used by infrastructure as code tools. The **Client** column marks solutions that use a client installed on nodes to configure. ✓ is used when it depends on the implementation. Stateful (resp. stateless) engines are marked ✓ (resp. ✗) in the column **State**. The **Communication** column indicates how the solutions communicate with nodes to configure. Finally, the **Reconciliation** column indicates the trigger mechanism for the reconfiguration process.

Push-based stateful and agentless provisioning. Azure Resource Manager, CloudFormation, Heat, Pulumi and Terraform follows almost the same execution model. Because provisioning tools are responsible for the initial creation of infrastructure, they necessarily adopt an agentless and push-based model; Since the target resources do not exist before the first deployment, no client or master/agent relationship can be established. Instead, these solutions push instructions directly to cloud APIs, which will provision resources. While this push is made manually, Azure Resource Manager, CloudFormation, and Heat allow scaling actions to be event-driven. These solutions rely on dedicated monitoring services, respectively Amazon CloudWatch, Azure Monitor, and Aodh (or Ceilometer), to detect when specific performance thresholds are reached. Once these services identify such an event, they push a signal to trigger the predefined scaling instructions. Besides, all provisioning solutions are stateful as they maintain a trace of resources currently managed. The storage of this state differs by provider:

- Azure Resource Manager, CloudFormation, and Heat manage state within the cloud platform. For Azure Resource Manager, the state is fully managed internally by Azure on Resource Group container. CloudFormation and Heat store the state in an internal database. The state is automatically updated during deployment operations.
- Pulumi and Terraform store state in an external file (respectively on a JSON file and on a 'terraform.tfstate' file). This file is updated after a successful execution or when an update is manually requested. While they do not support self-healing, they manage drift as they compare the recorded state against the live infrastructure to identify changes and propose a reconfiguration plan to reach the target state. This operation must be triggered manually by end users.

Push-based agentless configuration management. Ansible does not require a specific agent or daemon to be installed and continuously running on the target nodes in order to manage the configurations. Instead,

Ansible relies on SSH and Python, which have to be installed on distant nodes. Such a tool is thus defined as “agentless”, and necessarily adopts a push-based strategy for triggering reconfiguration action on distant nodes. For doing so, Ansible uses the SSH protocol to communicate from the control node (where Ansible is run) to the managed nodes. Consequently, the reconfiguration process is manual and occurs only when the user explicitly executes an Ansible script (i.e., playbook) from the control node. Furthermore, Ansible is fundamentally stateless; it does not maintain a persistent global record or external file to track the state of managed nodes. When executing a playbook, a set of information on nodes, namely facts, is retrieved and updated for the current execution only. Furthermore, even if the modules called within tasks expose a declarative programming style, they do not store state; instead, modules are executed locally on nodes and determine whether the module should be called or not (e.g., whether a file already exists with the same checksum before executing a copy).

Pull-based agent-driven configuration management. CFEngine, Chef, and Puppet rely on persistent agents that periodically pull and apply configuration policies. These tools maintain a state on a central master node: the Policy Hub for CFEngine, the Chef Infra Server for Chef, and the Puppet Server for Puppet. This master node stores the target configuration and state metadata for all managed nodes. In CFEngine, the state is managed via a distributed, self-healing mechanism where each *agent* (named *cf-agent*) running on a managed node periodically evaluates the policy against the local environment. While the master provides updates, the decision-making and state enforcement happen locally. Within Puppet, desired states are periodically pulled and applied by the *Puppet agent* running on each node. Chef follows a similar pull-based model: the Chef Infra Client runs on each managed node and, during each execution, gathers local system attributes to send to the Chef Infra Server. The server then sends back the relevant policy, and the client applies it locally to ensure the node converges to the target state. By default, the changes must be manually triggered, but periodic application can be set up on the master side.

Event-driven configuration. Unlike Ansible, which operates in a push-based model, and Puppet, Chef, and CFEngine, which follow a pull-based approach, SaltStack adopts a hybrid execution model. It relies on a central master node (the *Salt Master*) and persistent agents installed on the distant nodes, known as *minions*. Communication is handled via a high-speed bus (ZeroMQ), allowing the master to push commands to thousands of minions simultaneously, while minions can also pull their configuration from the master upon startup or at regular intervals. The state is managed centrally on the master through *state files*, which serve as a record of desired system properties. However, minions also maintain local state through *grains* (local system discovery) and *pillars* (secure variables distributed by the master). The reconfiguration process in SaltStack can be triggered manually by pushing a command from the master to specific minions. It can be configured to run periodically, where minions pull and apply the state at set intervals to prevent configuration drift. Finally, it supports an event-driven mode: *beacons* running on the minion monitor local metrics and emit events to the master’s *reactor*, which can automatically trigger a reconfiguration in response to real-time system changes.

Cluster management. Kubernetes follows a master/worker architecture. The master notably stores and manages the state of the resources. The state is stored in an ETCD distributed key-value storage with a REST API on top of it, known as *APIServer*. Hence, the desired state and the real state of the resources are stored centrally on the master side. Any change to these states requires a request to the *APIServer* that handles lock mechanisms. Kubernetes implements an event-driven strategy through resource version notifications. The master also hosts the *scheduler* component responsible for placement strategies on worker nodes, and the *controllers* responsible for continuously reconciling the state of resources on nodes. Both implement *watch* to *APIServer*. The worker nodes host the *kubelet* agents that request the *APIServer* to pull the required local changes. On worker nodes, *kubelet* is also responsible for restarting crashed containers without interactions with controllers or *APIServer*. Nomad also follows a master/worker architecture. Similarly to Kubernetes, the master is responsible for scheduling jobs on worker nodes and storing a consistent state of the cluster and its resources. The master centralizes supervision metrics of nodes and resources and takes new decisions if something changes or if new workloads are requested. Worker nodes pull their actions by requesting the

master. Note that both Kubernetes and Nomad support multi-masters deployment for high availability and guarantee consistency of the cluster state through distributed algorithms (i.e., Raft).

Hybrid approaches for an end-to-end management. During the provisioning phase, Juju instantiates a *master server* as a living controller to maintain a persistent `master` state in a central database (with MongoDB). Upon resource instantiation, a *client agent* (the Juju Machine Agent) is automatically installed on the managed nodes via a *push-based* initialization. Once the environment is established, the reconciliation model shifts to a *pull-based* mechanism: the agents use *watchers* (based on the observer pattern) to monitor the `master` state for changes. When a reconfiguration is triggered, agents automatically pull the delta to enforce the new state. Furthermore, the reconciliation is also processed with a *periodic* check.

Implementation-dependent end-to-end orchestration. As a specification standard, the execution model for TOSCA is implementation-dependent. While most solutions instantiate a *master server* to manage the deployment lifecycle, the use of a client depends on the adopted backend implementation. For instance, Cloudify typically adopts a *stateful, agent-based* approach where a *client agent* is installed on provisioned resources to facilitate a *pull-based* reconciliation. Conversely, orchestrators like xOpera favor an *agentless, push-based* model, interacting directly with target resources via remote execution protocols without resident software. As a consequence of the multiple existing implementations, TOSCA-based solutions can exhibit different mechanisms for reconciliation. For instance, xOpera supports only *manual* reconciliation, whereas Unfurl extends this with support for *periodic* checks. In contrast, platforms like Alien4Cloud or Cloudify enable *fully programmable* reconciliation.

7. Related work

There are attempts at classifying Infrastructure as Code (IaC) solutions. First, we discuss what technologies have been discussed across the different surveys about IaC, compared to ours. Then, we discuss what characteristics have been discussed compared to our approach.

7.1. Studied Infrastructure as Code solutions

	[37] (2018)	[38] (2019)	[39] (2020)	[5] (2023)	[40] (2025)	[41] (2025)	[42] (2025)	[43] (2025)	Our proposal (2026)
Azure Resource Manager	✗	✗	✓	✗	✗	✗	✗	✗	✓
CloudFormation	✓	✗	✓	✓	✗	✗	✓	✓	✓
Heat	✗	✗	✓	✗	✗	✗	✗	✓	✓
Omnia	✗	✓	✗	✗	✗	✗	✗	✗	✗
Pulumi	✗	✗	✗	✗	✓	✓	✓	✓	✓
Terraform	✗	✗	✓	✓	✓	✓	✓	✓	✓
Ansible	✗	✓	✓	✓	✓	✓	✓	✓	✓
CFEngine	✓	✗	✓	✗	✗	✗	✗	✗	✓
Charon	✗	✓	✗	✗	✗	✗	✗	✗	✗
Chef	✗	✓	✓	✓	✓	✓	✗	✓	✓
Puppet	✓	✓	✓	✓	✓	✓	✗	✓	✓
SaltStack	✗	✗	✓	✓	✗	✓	✗	✗	✓
Kubernetes	✗	✗	✓	✗	✗	✗	✗	✗	✓
Nomad	✗	✗	✗	✗	✗	✗	✗	✗	✓
DOML	✗	✗	✗	✗	✗	✗	✗	✓	✗
Juju	✓	✓	✓	✗	✗	✗	✗	✗	✓
TOSCA	✓	✗	✓	✗	✗	✗	✗	✓	✓
SmartFrog	✓	✗	✗	✗	✗	✗	✗	✗	✗

Table 4: Tools considered in the various surveys on IaC. ✓ is used when a tool is discussed in the survey while ✗ means it is not.

Table 4 summarizes the tools studied across surveys on infrastructure as code in the literature. Overall, our survey covers most of the established tools while also including under-represented ones such as Azure Resource Manager, Heat, or CFEngine.

Contrary to some, we intentionally exclude certain solutions based on their ecosystem. Charon [44] (now NixOps) is omitted due to its coupling with the NixOS [45] distribution and limited industrial adoption. Omnia [46] and SmartFrog [20] have no significant updates since 2017. Furthermore, we exclude DOML [47] because it is not an infrastructure as code tool but rather a high-level modeling approach designed to generate code for existing execution engines (Terraform and Ansible). Note that we classified the excluded tools according to our definition of capability.

Table 4 highlights several shifts in the popularity of infrastructure as code solutions in research. First, earlier surveys focused on cloud-specific provisioning, whereas recent studies prioritize *cloud-agnostic* solutions. Second, while Ansible, Chef, and Puppet appear in nearly all studies, older standards like CFEngine have disappeared from recent literature. Third, most existing surveys exclude *cluster managers* (e.g., Kubernetes, Nomad). They are traditionally categorized as orchestration platforms rather than infrastructure as code engines. Finally, Juju and TOSCA are frequently set aside due to their end-to-end nature, making them difficult to classify. Note that we intentionally do not give details about the TOSCA implementation studied in related surveys. They only consider the model, and do not study the notable differences of the implementations as we did in Section 6..

7.2. Classes for Infrastructure as Code

	[37] (2018)	[39] (2020)	[5] (2023)	[40] (2025)	[43] (2025)	Our proposal (2026)
Capability	✗	~	✓	✗	✓	✓
Adaptation	✗	✗	✗	✗	✗	✓
Target	✓	✓	✓	✗	✓	✓
Lang. construction	✗	~	✓	✓	✓	✓
Lang. logic style	✓	✗	✓	✓	✓	✓
Client agent	✓	✗	✓	✓	✓	✓
State management	✗	✗	✓	✗	✗	✓
Communication	✗	✗	✓	✓	✗	✓
Reconciliation	✗	✗	✗	✗	✗	✓
Ecosystem and Community	✓	✗	✗	✗	✓	✓
Terminology	✓	~	~	~	✗	✓
Mutability	✗	✗	✓	~	✓	~
Centralization	✓	✗	✗	✗	✗	~
Ease of use	✗	✗	✓	✓	✗	~

Table 5: Classes and aspects of IaC considered in the different surveys. ✓ is used when a class is concretely discussed in the survey; ~ means the class is barely discussed; and ✗ means the class is not mentioned.

Only a few works proposed a systematic classification of IaC tools. Table 5 summarizes the classes studied across surveys on infrastructure as code in the literature. We exclude works from our class-mapping comparison due to misalignments in scope or methodology. Rahman et al. [38] focus on the academic research landscape rather than a technical metamodel of solutions. Varma Indukuri [42] provides a general discussion without a formal mapping of engine characteristics. The study by Bali and Walila [48] is omitted for being too coarse-grained, comparing only three tools across broad qualitative metrics (efficiency, reliability) without technical class definitions. Simhandl and Zdun’s work [49] is focused on *Infrastructure from Code* (IfC); however, we consider IfC as a distinct paradigm that integrates infrastructure into application logic, whereas this survey focuses on infrastructure as code solutions. Finally, we excluded Mensah’s survey [41] since it is published in German.

While existing surveys primarily focus on tool intention and general usage, internal mechanisms such as reconciliation and behavioral models are rarely discussed. Our work distinguishes itself by providing specific nuance regarding adaptation strategies in cluster management and the granularity of reconciliation loops. From a usage perspective, the current literature focuses almost exclusively on end-users and not on the extensibility opportunities available to developers. As we do in Section 5., [37] and [39] acknowledge the existence of SDKs for platforms like AWS, but do not provide any details. This aspect will be discussed in a future survey.

Other aspects have been explored in the literature. Weerasiri et al. [37], consider the ecosystem as a taxonomic class to indicate the level of community support through blogs, forums, and repositories. While we address community contributions qualitatively in Section 3.2., we do not treat these as classification features. Similarly, [37] sets the terminology as a class. To ensure clarity, we dedicate significant focus to the underlying terminology of the solutions, but we intentionally exclude this class from our feature model and classification. Furthermore, we disagree with their characterization of centralization versus decentralization. In our view, what is often called “decentralization” in solutions is actually a distribution of computation or execution rather than a true decentralization. All existing industrial solutions rely on a central controller or master state, contrary to recent research efforts toward decentralized reconfiguration [50, 51]. Finally, some works discuss the “ease of use” of various solutions, a metric we find too arbitrary for a technical mapping. We limit our discussion to the objective requirements of which daemons must be executed and their physical location. Quantifying usability would require a dedicated, large-scale longitudinal study with a broad panel of infrastructure as code practitioners, which remains outside the scope of this architectural analysis. Finally, we did not discuss mutability

in detail. Pahl et al. [43] categorize solutions like Heat, Pulumi, Terraform, CloudFormation, and TOSCA as immutable, while Ozdogan et al. [5] consider CloudFormation and Terraform mutable. As mentioned in Section 5., these tools are inherently mutable but can be forced to behave immutably through explicit configuration.

Overall, existing surveys on infrastructure as code either do not target a general audience or lack a truly systematic approach. [5] offers the most rigorous classification framework, but is limited by its publication in a national journal. [37] covers only a small set of tools, while [39] prioritizes deployment over a dedicated IaC architectural perspective. More recent efforts like [40] focus on popular engines with highly similar internal behaviors without proposing a concrete classification, and [43] provides only a brief excerpt of a classification intended to highlight open challenges regarding AI integration [52].

7.3. Quality assurance in Infrastructure as Code

Additional work has examined IaC from a quality-improvement perspective, proposing defect categorizations. Several studies have explored practitioner-reported challenges through surveys, mining online forums, and gray literature, identifying recurring issues such as configuration errors, deployment failures, testing difficulties, and maintainability problems [53, 54, 55, 56, 57]. Additional research has studied IaC technology- and tooling-oriented defects. These studies have proposed classifications (defect categories or bug taxonomies) based on empirical analyses of scripts and systems [43, 58, 59, 60, 61]. Finally, systematic surveys have consolidated these efforts by organizing defects around static analysis capabilities and configuration quality concerns [62, 63].

8. Conclusion

This survey provides a comprehensive classification of the infrastructure as code solution, highlighting a large spectrum of classes like capabilities and behavioral models. The diversity of these solutions underscores the difficulty in strictly framing infrastructure as code within a single definition. Instead, the field presents a multi-faceted paradigm where the choice of a tool involves trade-offs.

Looking toward the future, the emergence of *Infrastructure from Code* (IfC) [17] presents a significant challenge: as applications are heterogeneous, a “one size fits all” infrastructure approach is not possible. We argue that the next major challenge lies in establishing a mapping between application characteristics and the optimal behavioral model based on specific performance and reliability metrics. Furthermore, even once a best-fit solution is selected, operational challenges such as resource sizing persist. To address these, AI-driven optimizations are emerging as essential tools for assisting practitioners in the correct and efficient use of infrastructure as code technologies [52].

Finally, the literature about infrastructure as code engines tends to adopt principles from *Component-Based Software Engineering* (CBSE) [64]. To address the cross-DevOps needs, academic efforts, such as Concerto-D [65], Ballet [50], and μ S [51], have designed new models based on decentralization. In addition, formal works [24, 66, 67, 68] on IaC have started to pave the way for more robust solutions.

9. Bibliography

- [1] Rosemary Wang. *Infrastructure as Code, Patterns and Practices: With Examples in Python and Terraform*. Simon and Schuster, 2022.
- [2] Xudong Sun, Wenjie Ma, Jiawei Tyler Gu, Zicheng Ma, Tej Chajed, Jon Howell, Andrea Lattuada, Oded Padon, Lalith Suresh, Adriana Szekeres, and Tianyin Xu. Anvil: verifying liveness of cluster management controllers. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation, OSDI'24, USA*, 2024. USENIX Association.
- [3] Bingzhe Liu, Gangmuk Lim, Ryan Beckett, and P. Brighten Godfrey. Kivi: verification for cluster management. In *Proceedings of the 2024 USENIX Conference on Usenix Annual Technical Conference, USENIX ATC'24, USA*, 2024. USENIX Association.

- [4] Ruben Opdebeeck, Ahmed Zerouali, and Coen De Roover. *Infrastructure-as-Code Ecosystems*, pages 215–245. Springer International Publishing, Cham, 2023.
- [5] Erdal Özdoğan, Onur Ceran, and Mutlu ÜSTÜNDAĞ. Systematic analysis of infrastructure as code technologies. *Gazi University Journal of Science Part A: Engineering and Innovation*, 10, 11 2023.
- [6] V. Subbiah. Understanding infrastructure as code (iac), 2024. Accessed: 2025-04-22.
- [7] Splunk Inc. What is infrastructure as code? an introduction to iac, 2023. Accessed: 2025-04-22.
- [8] Wikipedia contributors. Infrastructure as code — wikipedia entry. https://en.wikipedia.org/wiki/Infrastructure_as_code, 2025. Accessed: 2025-04-22.
- [9] Mark Burgess. A site configuration engine. *Computing Systems*, 8(3):309–337, 1995.
- [10] CFEngine - Automated Configuration Management, 1993. Accessed: 2025-06-04.
- [11] Chef, 2025. Accessed: 2025-06-04.
- [12] Red hat ansible automation platform, 2025. Accessed: 2025-06-04.
- [13] Aws cloudformation, 2025. Accessed: 2025-06-04.
- [14] Yevgeniy Brikman. *Terraform: Up and Running*. " O'Reilly Media, Inc.", 2022.
- [15] Charles Anderson. Docker. *IEEE software*, 32(3):102–c3, 2015.
- [16] Brendan Burns, Joe Beda, Kelsey Hightower, and Lachlan Evenson. *Kubernetes: up and running*. " O'Reilly Media, Inc.", 2022.
- [17] Itzhak Aviv, Ruti Gafni, Sofia Sherman, Berta Aviv, Asher Sterkin, and Etzik Bega. Infrastructure from code: The next generation of cloud lifecycle automation. *IEEE Software*, 40(1):42–49, 2022.
- [18] Hashicorp cloud platform, 2025. Accessed: 2026-02-13.
- [19] Infrastructure manager, 2025. Accessed: 2025-06-04.
- [20] Patrick Goldsack, Julio Guijarro, Antonio Lain, Guillaume Mecheneau, Paul Murray, and Peter Toft. Smartfrog: Configuration and automatic ignition of distributed applications. *HP OVUA*, 2003.
- [21] John A Hewson, Paul Anderson, and Andrew D Gordon. A declarative approach to automated configuration. In *26th Large Installation System Administration Conference (LISA 12)*, pages 51–66, 2012.
- [22] Fabien Hermenier, Julia Lawall, and Gilles Muller. Btrplace: A flexible consolidation manager for highly available applications. *IEEE Trans. Dependable Secur. Comput.*, 10(5):273–286, September 2013.
- [23] Baptiste Jonglez, Matthieu Simonin, Jolan Philippe, and Sidi Mohammed Kaddour. Multi-provider capabilities in enoslib: driving distributed system experiments on the edge-to-cloud continuum. In *IFIP International Conference on Distributed Applications and Interoperable Systems*, pages 25–42. Springer, 2025.
- [24] Maverick Chardet, Hélène Coullon, and Simon Robillard. Toward safe and efficient reconfiguration with concerto. *Science of Computer Programming*, 203:102582, 2021.
- [25] (r)?ex, 2025. Accessed: 2026-02-13.
- [26] Azure resource manager, 2025. Accessed: 2025-06-04.
- [27] Openstack heat, 2025. Accessed: 2025-06-04.
- [28] Juju, 2025. Accessed: 2025-06-04.

- [29] Nomad, 2025. Accessed: 2025-06-04.
- [30] Pulum, 2025. Accessed: 2025-06-04.
- [31] What is puppet, 2014. Accessed: 2025-06-04.
- [32] Isconf, 2002. Accessed: 2025-11-19.
- [33] Moshe Zadka. Saltstack, 2019.
- [34] Opentofu, 2025. Accessed: 2025-09-16.
- [35] Tobias Binz, Uwe Breitenbücher, Florian Haupt, Oliver Kopp, Frank Leymann, Alexander Nowak, and Sebastian Wagner. Opentosca—a runtime for toasca-based cloud applications, 2013.
- [36] Miles Stötzner, Steffen Becker, Lukas Harzenetter, Frank Leymann, Benjamin Weder, Uwe Breitenbücher, Oliver Kopp, Kevin Klein, and Jacopo Soldani. A systematic technology review of general-purpose open-source toasca orchestrators. In *Proceedings of the IEEE/ACM 16th International Conference on Utility and Cloud Computing, UCC '23*, New York, NY, USA, 2024. Association for Computing Machinery.
- [37] Denis Weerasiri, Moshe Chai Barukh, Boualem Benatallah, Quan Z. Sheng, and Rajiv Ranjan. A Taxonomy and Survey of Cloud Resource Orchestration Techniques. 50(2):1–41.
- [38] Akond Rahman, Rezvan Mahdavi-Hezaveh, and Laurie Williams. A systematic mapping study of infrastructure as code research. *Information and Software Technology*, 108:65–77, 2019.
- [39] Michael Wurster, Uwe Breitenbücher, Michael Falkenthal, Christoph Krieger, Frank Leymann, Karoline Saatkamp, and Jacopo Soldani. The essential deployment metamodel: a systematic review of deployment automation technologies. *SICS Software-Intensive Cyber-Physical Systems*, 35(1):63–75, 2020.
- [40] Sergei Bykov and Vladimir Shiboldenkov. Infrastructure-as-code approach for it-infrastructure. pages 1–6, 04 2025.
- [41] Jonah-Myron Mensah. Evaluation von infrastructure as code tools für cloud-infrastruktur, 2025.
- [42] Ajay Varma Indukuri. Infrastructure as code: A paradigm shifts in cloud resource management and deployment automation. 2025.
- [43] Claus Pahl, Niyazi Gunduz, Övgüm Sezen, Ali Ghamgosar, and Nabil El Ioini. Infrastructure as code: Technology review and research challenges. pages 151–158, 2025.
- [44] Eelco Dolstra, Rob Vermaas, and Shea Levy. Charon: Declarative provisioning and deployment. In *2013 1st International Workshop on Release Engineering (RELENG)*, pages 17–20. IEEE, 2013.
- [45] Eelco Dolstra and Andres Löb. Nixos: A purely functional linux distribution. In *Proceedings of the 13th ACM SIGPLAN international conference on Functional programming*, pages 367–378, 2008.
- [46] Marco Miglierina and Damian A Tamburri. Towards omnia: A monitoring factory for quality-aware devops. In *Proceedings of the 8th ACM/SPEC on International Conference on Performance Engineering Companion*, pages 145–150, 2017.
- [47] Michele Chiari, Bin Xiang, Sergio Canzoneri, Galia Novakova Nedeltcheva, Elisabetta Di Nitto, Lorenzo Blasi, Debora Benedetto, Laurentiu Niculut, and Igor Škof. Doml: A new modeling approach to infrastructure-as-code. *Information Systems*, 125:102422, 2024.
- [48] Milandeep Kour Bali and Ranjan Walia. Enhancing efficiency through infrastructure automation: An in-depth analysis of infrastructure as code (iac) tools. In *2023 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS)*, pages 857–863, 2023.

- [49] Georg Simhandl and Uwe Zdun. Cloud programming languages and infrastructure from code: An empirical study. In *Proceedings of the 17th ACM SIGPLAN International Conference on Software Language Engineering, SLE '24*, page 143–156, New York, NY, USA, 2024. Association for Computing Machinery.
- [50] Jolan Philippe, Antoine Omond, H          , Charles Prud'Homme, and Issam Ra    . Fast choreography of cross-devops reconfiguration with ballet: A multi-site openstack case study. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 1–11, 2024.
- [51] Daniel Sokolowski, Pascal Weisenburger, and Guido Salvaneschi. Automating serverless deployments for devops organizations, 2021.
- [52] Claus Pahl,        Can Sezen, and Florian Hofer. Artificial intelligence for infrastructure-as-code—a systematic literature review. *Electronics*, 15(4), 2026.
- [53] Michele Guerriero, Martin Garriga, Damian A. Tamburri, and Fabio Palomba. Adoption, support, and challenges of infrastructure-as-code: Insights from industry. In *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 580–589, 2019.
- [54] Indika Kumara, Mart     Garriga, Angel Urbano Romeu, Dario Di Nucci, Fabio Palomba, and Willem-Jan Tamburri, Damian Andrew and van den Heuvel. The do's and don'ts of infrastructure code: A systematic gray literature review. *Information and Software Technology*, 137:106593, 2021.
- [55] Mahi Begoug, Narjes Bessghaier, Ali Ouni, Eman Abdullah AlOmar, and Mohamed Wiem Mkaouer. What do infrastructure-as-code practitioners discuss: An empirical study on stack overflow. In *2023 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pages 1–12, 2023.
- [56] Mohammed Mehedi Hasan, Farzana Ahamed Bhuiyan, and Akond Rahman. Testing practices for infrastructure as code. *LANGETI 2020*, page 7–12, New York, NY, USA, 2020. Association for Computing Machinery.
- [57] David Reis, Bruno Piedade, Filipe F. Correia, Jo     Pedro Dias, and Ademar Aguiar. Developing docker and docker-compose specifications: A developers' survey. *IEEE Access*, 10:2318–2329, 2022.
- [58] Akond Rahman and Tushar Sharma. Lessons from research to practice on writing better quality puppet scripts. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 63–67, 2022.
- [59] Julian Schwarz, Andreas Steffens, and Horst Lichter. Code smells in infrastructure as code. In *2018 11th International Conference on the Quality of Information and Communications Technology (QUATIC)*, pages 220–228, 2018.
- [60] Georgios-Petros Drosos, Thodoris Sotiropoulos, Georgios Alexopoulos, Dimitris Mitropoulos, and Zhen-dong Su. When your infrastructure is a buggy program: Understanding faults in infrastructure as code ecosystems. *Proc. ACM Program. Lang.*, 8(OOPSLA2), October 2024.
- [61] Boubacar Diarra, Karine Guillouard, Meryem Ouzzif, Philippe Merle, and Jean-Bernard Stefani. In-depth analysis of kubernetes manifest verification tools for robust cnf deployment. In *2024 27th Conference on Innovation in Clouds, Internet and Networks (ICIN)*, pages 17–24, 2024.
- [62] Michele Chiari, Michele De Pascalis, and Matteo Pradella. Static analysis of infrastructure as code: a survey. In *2022 IEEE 19th International Conference on Software Architecture Companion (ICSA-C)*, pages 218–225, 2022.
- [63] Pandu Ranga Reddy Konala, Vimal Kumar, and David Bainbridge. Sok: Static configuration analysis in infrastructure as code scripts. In *2023 IEEE International Conference on Cyber Security and Resilience (CSR)*, pages 281–288, 2023.
- [64] H          , Ludovic Henrio, Fr            , and Simon Robillard. Component-based distributed software reconfiguration:a verification-oriented survey. *ACM Comput. Surv.*, 56(1), August 2023.

- [65] Farid Arfi, Hélène Coullon, Frédéric Loulergue, Jolan Philippe, and Simon Robillard. An overview of the decentralized reconfiguration language concerto-d through its maude formalization. *Electronic Proceedings in Theoretical Computer Science*, 414:21–38, December 2024.
- [66] Roberto Di Cosmo, Jacopo Mauro, Stefano Zacchiroli, and Gianluigi Zavattaro. Aeolus: A component model for the cloud. *Information and Computation*, 239:100–121, 2014.
- [67] Paul Anderson and Herry Herry. A formal semantics for the smartfrog configuration language. *Journal of Network and Systems Management*, 24:309–345, 2016.
- [68] Weili Fu, Roly Perera, Paul Anderson, and James Cheney. mupuppet: A declarative subset of the puppet configuration language. 74:1–27, 2017.

10. Liste des acronymes, graphiques et tableaux

10.1. Liste des acronymes et abréviations

AI	Artificial Intelligence
API	Application Programming Interface
ARM	Azure Resource Manager
AWS	Amazon Web Services
CBSE	Component-Based Software Engineering
CD	Continuous Deployment
CFN	CloudFormation
CI	Continuous Integration
CLI	Command Line Interface
DSL	Domain-Specific Language
EC2	Elastic Compute Cloud
GCP	Google Cloud Platform
GPL	General Purpose Language
gRPC	Google Remote Procedure Call
HCL	HashiCorp Configuration Language
HOT	Heat Orchestration Template
HTTP	Hypertext Transfer Protocol
IaC	Infrastructure as Code
IaaS	Infrastructure as a Service
IfC	Infrastructure from Code
JSON	JavaScript Object Notation
LSP	Language Server Protocols

OASIS Organization for the Advancement of Structured Information Standards

PaaS Platform as a Service

REST Representational State Transfer

SaaS Software as a Service

SSH Secure Shell

TOSCA Topology and Orchestration Specification for Cloud Applications

VM Virtual Machine

YAML YAML Ain't Markup Language

10.2. List of Tables

1	Capabilities of infrastructure as code tools	13
2	Characteristics of technologies from an end-user perspective.	16
3	Execution models used by infrastructure as code tools.	18
4	Tools considered in the various surveys on IaC.	21
5	Classes and aspects of IaC considered in the different surveys.	22

10.3. List of Figures

1	Global overview of tasks operated by infrastructure as code	6
2	Timeline of Infrastructure as Code capabilities appearance	8
3	Diagram of studied features in IaC technologies	11